



2.5 практика наверняка уже реализуется планомерно при работе, здесь отдельную не выделяем. Но на свой вкус можете что-нибудь показать Практика (0,7 ч.): отладка агента при неудачном выполнении задачи, анализ логов и корректировка промптов.

Что мы будем делать

Сейчас я хочу показать следующий уровень работы с Hermes. До этого агент уже умел пользоваться встроенным терминалом, файловыми инструментами и нашим RAG-поиском. Но теперь мы создадим три специализированных Tool.

Первый инструмент будет получать `git diff`. Второй будет выполнять поиск по содержимому проекта. Третий будет запускать линтер Ruff.

Важно, что агент не будет получать произвольную строку командной оболочки. Каждый Tool будет иметь заранее определённое назначение, конкретные аргументы и собственную проверку входных данных.

Это и есть одна из причин создавать отдельные инструменты вместо постоянного использования универсального терминала.

Открываем проект

Я запускаю Hermes Desktop и открываю наш Project:

```
dependency-lab
```

Рабочая директория должна быть корнем проекта `hermes-multiagent`.

```
hermes-multiagent
```

Справа открываю Terminal. Если он скрыт, нажимаю:

```
Ctrl + `
```

Проверяю путь:

```
Get-Location
```

Я ожидаю, что последний сегмент пути называется `hermes-multiagent`. Сохраняю текущий путь в переменную, чтобы команды не зависели от расположения проекта:

```
$projectRoot = (Get-Location).Path
```

Если открыт другой каталог, возвращаюсь в Project `dependency-lab` через интерфейс Hermes и повторяю проверку.

Устанавливаем Ruff

Для третьего инструмента нам понадобится линтер.

Чтобы зависимость среды разработки оставалась воспроизводимой, создаю в корне `dev-requirements.txt`:

```
ruff
```

В терминале выполняю:

```
.\.venv\Scripts\python.exe -m pip install -r dev-requirements.txt
```

Проверяю:

```
.\.venv\Scripts\python.exe -m ruff --version
```

Я должен получить номер установленной версии.

Теперь Ruff существует внутри виртуального окружения проекта и не зависит от глобальной установки Python.

Подготавливаем файл с ошибкой для линтера

Создаём небольшой учебный файл, чтобы `run_linter` гарантированно нашёл проблему.

В чате Hermes отправляю:

Создай файл:

`demo_codebase\quality\bad_example.py`

с указанным содержимым.

Другие файлы не изменяй.

Содержимое:

```
import os
```

```
def normalize_name(name: str) -> str:  
    return name.strip().lower()
```

Здесь импорт `os` не используется. Ruff должен обнаружить правило `F401`.

Пока ничего не исправляем.

Подготавливаем файл для git diff

Теперь создадим отдельный файл, относительно которого удобно показывать Git diff.

Отправляю Hermes:

Создай в корне проекта файл `dev_lab.py` со следующим содержимым:

```
def format_username(name: str) -> str:  
    return name.strip().lower()
```

Другие файлы не изменяй.

Теперь в Terminal выполняю:

```
git add dev_lab.py
```

Эта команда помещает исходную версию файла в Git index и создаёт staged baseline без отдельного коммита. Так мы сохраняем правило «одна практическая — один итоговый коммит» и всё равно можем сравнить исходную версию с рабочей.

Теперь изменим уже отслеживаемый файл.

В чате Hermes отправляю:

В файле `dev_lab.py` измени только возвращаемое значение функции так, чтобы пробелы в имени пользователя заменялись символом подчёркивания.

Другие файлы не изменяй.

Ожидаемая логика теперь будет связана с:

```
replace(" ", "_")
```

В Terminal проверяю:

```
git diff -- dev_lab.py
```

Diff должен существовать.

Поскольку baseline находится в index, обычный `git diff` показывает точное изменение между исходной и рабочей версиями. До итоговой фиксации `git diff --cached` показывает staged-добавление исходной версии `dev_lab.py`; это ожидаемо.

Этот diff позже будет получать наш собственный Hermes Tool.

Создаём каталог plugin

Теперь начинаем регистрацию инструментов.

В Terminal выполняю:

```
New-Item -ItemType Directory -Force .hermes\plugins\dev-tools
```

Проверяю:

```
Get-ChildItem .hermes\plugins
```

Помимо предыдущего `code-rag` должен появиться:

```
dev-tools
```

Project-local plugins в Hermes находятся в `.hermes/plugins/`, но по умолчанию не загружаются без явного разрешения для доверенного проекта. Кроме того, plugin после обнаружения нужно отдельно включить.

Создаём plugin.yaml

В чате Hermes отправляю:

Создай файл:

```
.hermes\plugins\dev-tools\plugin.yaml
```

строго со следующим содержимым.

```
name: dev-tools
version: "1.0.0"
description: Безопасные инструменты анализа Python-проекта для учебной работы
provides_tools:
  - git_diff
  - grep_search
```

```
- run_linter
python_runtime: external
```

Здесь мы объявили три инструмента, которые plugin должен зарегистрировать. Hermes сверяет заявленные возможности plugin с фактической регистрацией, в том числе через Plugin Doctor.

Создаём схемы инструментов

Теперь создаём файл, который особенно важен именно для LLM.

Отправляю:

Создай файл:

```
.hermes\plugins\dev-tools\schemas.py
```

строго со следующим содержимым.

```
"""Схемы инструментов, которые получает языковая модель."""

GIT_DIFF = {
    "name": "git_diff",
    "description": (
        "Показывает текущий Git diff проекта без изменения репозитория. "
        "Используй для проверки незакоммиченных изменений, строк изменений "
        "и read-only ревью; поддерживается обычный и staged diff."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "Необязательный путь внутри проекта; по умолчанию весь проект.",
            },
            "staged": {
                "type": "boolean",
                "description": "Если true, показать staged diff через git diff --cached.",
            },
        },
        "required": [],
        "additionalProperties": False,
    },
}

GREP_SEARCH = {
    "name": "grep_search",
    "description": (
```

```

        "Ищет текст или шаблон в отслеживаемых Git файлах без изменения проекта. "
        "Используй для поиска применений функций, классов, импортов и API."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "pattern": {
                "type": "string",
                "minLength": 1,
                "maxLength": 500,
                "description": "Непустая строка или шаблон для поиска.",
            },
            "path": {
                "type": "string",
                "description": "Необязательный путь внутри проекта для ограничения поиска.",
            },
            "max_results": {
                "type": "integer",
                "minimum": 1,
                "maximum": 100,
                "description": "Максимум строк результата; по умолчанию 30.",
            },
        },
        "required": ["pattern"],
        "additionalProperties": False,
    },
}

RUN_LINTER = {
    "name": "run_linter",
    "description": (
        "Запускает Ruff только в режиме статической проверки, без автоматических "
        "исправлений. Используй для оценки качества Python-кода после изменений."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "Необязательный путь к Python-файлу или каталогу внутри проек-
та.",
            },
        },
        "required": [],
        "additionalProperties": False,
    },
}

```

Я отдельно обращаю внимание студентов на поле `description`.

Это не комментарий для программиста. Именно это описание получает модель и на его основании решает, когда использовать Tool.

В официальной документации Hermes прямо отмечено, что `description` объясняет LLM назначение инструмента, а `parameters` задаёт аргументы вызова.

Создаём обработчики Tools

Теперь создаём код, который реально будет выполняться.

Отправляю Hermes:

Создай файл:

`.hermes\plugins\dev-tools\tools.py`

строго со следующим содержимым.

```
"""Безопасные обработчики инструментов разработки."""

from __future__ import annotations

import json
import logging
import os
import subprocess
import tempfile
import time
from dataclasses import dataclass
from logging.handlers import TimedRotatingFileHandler
from pathlib import Path
from typing import Any, BinaryIO

PLUGIN_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = PLUGIN_DIR.parents[2]
PYTHON_EXECUTABLE = PROJECT_ROOT / ".venv" / "Scripts" / "python.exe"
LOG_DIR = PROJECT_ROOT / "rag" / "logs"
LOG_FILE = LOG_DIR / "dev-tools.log"
LOG_RETENTION_SECONDS = 30 * 24 * 60 * 60
DIAGNOSTIC_LIMIT = 20_000

def _configure_logger() -> logging.Logger:
    """Настраивает журнал с ротацией и безопасной деградацией при ошибке."""

    logger = logging.getLogger("dev_tools_plugin")
    logger.setLevel(logging.INFO)
    logger.propagate = False
```

```

    if logger.handlers:
        return logger

    try:
        LOG_DIR.mkdir(parents=True, exist_ok=True)
        cutoff = time.time() - LOG_RETENTION_SECONDS
        for log_path in LOG_DIR.glob("dev-tools.log*"):
            if log_path.is_file() and log_path.stat().st_mtime < cutoff:
                log_path.unlink()
        handler = TimedRotatingFileHandler(
            LOG_FILE, when="midnight", backupCount=30, encoding="utf-8"
        )
        handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
        logger.addHandler(handler)
    except OSError:
        logger.addHandler(logging.NullHandler())
    return logger

LOGGER = _configure_logger()

def _json(data: dict[str, Any]) -> str:
    """Сериализует ответ инструмента с поддержкой русского текста."""

    return json.dumps(data, ensure_ascii=False)

def _error(message: str) -> str:
    """Возвращает согласованный JSON-ответ об ошибке."""

    return _json({"error": message})

def _validate_args(args: Any, operation: str) -> dict[str, Any] | None:
    """Принимает только объект аргументов Hermes."""

    if isinstance(args, dict):
        return args
    LOGGER.warning("operation=%s rejected=arguments_not_object", operation)
    return None

def _safe_relative_path(raw_path: str | None) -> tuple[Path | None, str | None]:
    """Возвращает путь внутри проекта, не допуская обход через ссылки и ``..``."""

    if raw_path is None:
        return None, None
    if not isinstance(raw_path, str):
        return None, "Параметр path должен быть строкой."

```

```

value = raw_path.strip()
if not value:
    return None, None

try:
    supplied = Path(value)
    if supplied.is_absolute() or supplied.drive or ".." in supplied.parts:
        return None, "Путь выходит за пределы проекта."
    root = PROJECT_ROOT.resolve()
    candidate = (root / supplied).resolve()
    relative = candidate.relative_to(root)
except (OSError, RuntimeError, ValueError):
    return None, "Путь выходит за пределы проекта."
return relative, None

def _read_limited(stream: BinaryIO, limit: int) -> tuple[str, bool]:
    """Читает временный файл в память не более ``limit + 1`` байта."""

    stream.seek(0)
    data = stream.read(limit + 1)
    truncated = len(data) > limit
    if truncated:
        data = data[:limit]
    return data.decode("utf-8", errors="replace"), truncated

@dataclass(frozen=True)
class _ProcessResult:
    """Ограниченный результат внешней программы."""

    returncode: int
    stdout: str
    stderr: str
    stdout_truncated: bool
    stderr_truncated: bool

def _run_bounded(command: list[str], timeout: int, stdout_limit: int = DIAGNOSTIC_LIMIT) ->
_ProcessResult:
    """Запускает программу без shell, сохраняя неограниченный вывод только во временный файл."""

    environment = dict(os.environ)
    environment["PYTHONUTF8"] = "1"
    with tempfile.TemporaryFile() as stdout_file, tempfile.TemporaryFile() as stderr_file:
        process = subprocess.run(
            command,
            cwd=str(PROJECT_ROOT),
            stdout=stdout_file,
            stderr=stderr_file,
            timeout=timeout,

```

```

        env=environment,
        shell=False,
        check=False,
    )
    stdout, stdout_truncated = _read_limited(stdout_file, stdout_limit)
    stderr, stderr_truncated = _read_limited(stderr_file, DIAGNOSTIC_LIMIT)
    return _ProcessResult(
        process.returncode, stdout, stderr, stdout_truncated, stderr_truncated
    )

def _log_finish(operation: str, started: float, outcome: str, **details: Any) -> None:
    """Записывает метаданные операции без содержимого diff и поискового шаблона."""

    detail_text = " ".join(f"{key}={value}" for key, value in details.items())
    LOGGER.info(
        "operation=%s outcome=%s duration_ms=%s %s",
        operation,
        outcome,
        round((time.monotonic() - started) * 1000),
        detail_text,
    )

def git_diff(args: dict[str, Any], **kwargs: Any) -> str:
    """Возвращает ограниченный обычный или staged Git diff."""

    del kwargs
    started = time.monotonic()
    values = _validate_args(args, "git_diff")
    if values is None:
        return _error("Аргументы инструмента должны быть JSON-объектом.")
    staged = values.get("staged", False)
    if not isinstance(staged, bool):
        LOGGER.warning("operation=git_diff rejected=invalid_staged")
        return _error("Параметр staged должен быть логическим значением.")
    relative_path, path_error = _safe_relative_path(values.get("path"))
    if path_error:
        LOGGER.warning("operation=git_diff rejected=path")
        return _error(path_error)

    command = ["git", "diff", "--no-ext-diff", "--no-textconv"]
    if staged:
        command.append("--cached")
    if relative_path is not None:
        command.extend(["--", str(relative_path)])
    try:
        result = _run_bounded(command, timeout=30)
    except subprocess.TimeoutExpired:
        _log_finish("git_diff", started, "timeout", staged=staged)
        return _error("git diff превысил лимит времени.")

```

```

except (OSError, ValueError):
    _log_finish("git_diff", started, "execution_error", staged=staged)
    return _error("Не удалось выполнить git diff.")
if result.returncode != 0:
    _log_finish("git_diff", started, "exit_error", exit_code=result.returncode)
    return _error("git diff завершился с ошибкой.")

_log_finish("git_diff", started, "ok", staged=staged, path=relative_path or ".")
return _json({
    "staged": staged,
    "path": str(relative_path) if relative_path is not None else ".",
    "diff": result.stdout or "Изменений нет.",
    "truncated": result.stdout_truncated,
})

def grep_search(args: dict[str, Any], **kwargs: Any) -> str:
    """Ищет совпадения ``git grep`` с ограниченным выводом."""

    del kwargs
    started = time.monotonic()
    values = _validate_args(args, "grep_search")
    if values is None:
        return _error("Аргументы инструмента должны быть JSON-объектом.")
    pattern = values.get("pattern")
    if not isinstance(pattern, str) or not pattern.strip():
        LOGGER.warning("operation=grep_search rejected=pattern")
        return _error("Параметр pattern не может быть пустым.")
    pattern = pattern.strip()
    if len(pattern) > 500:
        LOGGER.warning("operation=grep_search rejected=pattern_length")
        return _error("Поисковый шаблон слишком длинный.")
    raw_max_results = values.get("max_results", 30)
    if isinstance(raw_max_results, bool) or not isinstance(raw_max_results, int):
        LOGGER.warning("operation=grep_search rejected=max_results_type")
        return _error("Параметр max_results должен быть целым числом от 1 до 100.")
    if not 1 <= raw_max_results <= 100:
        LOGGER.warning("operation=grep_search rejected=max_results_range value=%s",
raw_max_results)
        return _error("Параметр max_results должен быть в диапазоне от 1 до 100.")
    relative_path, path_error = _safe_relative_path(values.get("path"))
    if path_error:
        LOGGER.warning("operation=grep_search rejected=path")
        return _error(path_error)

    command = ["git", "grep", "-n", "-I", "-e", pattern, "--"]
    if relative_path is not None:
        command.append(str(relative_path))
    try:
        result = _run_bounded(command, timeout=30)
    except subprocess.TimeoutExpired:

```

```

        _log_finish("grep_search", started, "timeout", max_results=raw_max_results)
        return _error("Поиск превысил лимит времени.")
    except (OSError, ValueError):
        _log_finish("grep_search", started, "execution_error", max_results=raw_max_results)
        return _error("Не удалось выполнить поиск.")
    if result.returncode not in {0, 1}:
        _log_finish("grep_search", started, "exit_error", exit_code=result.returncode)
        return _error("git grep завершился с ошибкой.")

    lines = result.stdout.splitlines()
    if result.stdout_truncated and lines and not result.stdout.endswith(("\\n", "\\r")):
        lines.pop()
    matches = lines[:raw_max_results]
    truncated = result.stdout_truncated or len(lines) > raw_max_results
    response: dict[str, Any] = {
        "path": str(relative_path) if relative_path is not None else ".",
        "shown_matches": len(matches),
        "matches": matches,
        "truncated": truncated,
    }
    if not result.stdout_truncated:
        response["total_matches"] = len(lines)
    _log_finish("grep_search", started, "ok", path=relative_path or ".",
max_results=raw_max_results)
    return _json(response)

def run_linter(args: dict[str, Any], **kwargs: Any) -> str:
    """Запускает Ruff без изменений файлов и возвращает результат проверки."""

    del kwargs
    started = time.monotonic()
    values = _validate_args(args, "run_linter")
    if values is None:
        return _error("Аргументы инструмента должны быть JSON-объектом.")
    relative_path, path_error = _safe_relative_path(values.get("path"))
    if path_error:
        LOGGER.warning("operation=run_linter rejected=path")
        return _error(path_error)
    if not PYTHON_EXECUTABLE.is_file():
        LOGGER.error("operation=run_linter outcome=python_not_found")
        return _error("Не найден Python виртуального окружения проекта.")

    target = str(relative_path) if relative_path is not None else "."
    command = [
        str(PYTHON_EXECUTABLE), "-m", "ruff", "check", target,
        "--output-format", "concise", "--no-cache",
    ]
    try:
        result = _run_bounded(command, timeout=60)
    except subprocess.TimeoutExpired:

```

```
_log_finish("run_linter", started, "timeout", path=target)
return _error("Ruff превысил лимит времени.")
except (OSError, ValueError):
    _log_finish("run_linter", started, "execution_error", path=target)
    return _error("Не удалось запустить Ruff.")
if result.returncode >= 2:
    _log_finish("run_linter", started, "exit_error", exit_code=result.returncode,
path=target)
    return _error("Ruff завершился с ошибкой выполнения.")

_log_finish("run_linter", started, "ok" if result.returncode == 0 else "lint_found",
exit_code=result.returncode, path=target)
diagnostics = result.stdout or result.stderr
return _json({
    "passed": result.returncode == 0,
    "exit_code": result.returncode,
    "path": target,
    "diagnostics": diagnostics,
    "truncated": result.stdout_truncated or result.stderr_truncated,
})
```

Я обращаю внимание на несколько моментов.

Мы не используем `shell=True`. Команды передаются в `subprocess.run()` как массив аргументов. Это уменьшает риск того, что значение параметра превратится в дополнительную shell-команду.

Мы также проверяем путь и не позволяем инструменту работать за пределами корня проекта.

Вывод внешних команд сначала направляется во временные файлы, а в память читается только ограниченный объём. Поэтому большой diff или диагностический вывод не приводит к неограниченному расходу памяти.

`git_diff` отключает внешние diff-команды и `textconv`. `run_linter` не использует `--fix` и запускает Ruff с `--no-cache`, поэтому проверка не изменяет исходники и не создаёт `.ruff_cache`. Код возврата Ruff `1` означает найденные замечания, а код `2` и выше считается ошибкой выполнения инструмента.

Операции записывают только диагностические метаданные в `rag/logs/dev-tools.log`. Журналы ротируются ежедневно и хранятся не более 30 дней; поисковые шаблоны и содержимое diff в журнал не попадают.

И наконец, каждый обработчик возвращает JSON даже при ошибке. Такой формат соответствует контракту Hermes plugin tools.

Регистрируем инструменты

Теперь создаём последний файл plugin.

Отправляю:

Создай файл:

`.hermes\plugins\dev-tools\__init__.py`

строго со следующим содержимым.

```
"""Регистрация инструментов плагина dev-tools."""

from typing import Any

from . import schemas, tools

def register(ctx: Any) -> None:
    """Регистрирует безопасные инструменты анализа в Hermes."""

    ctx.register_tool(
        name="git_diff",
        toolset="dev_tools",
        schema=schemas.GIT_DIFF,
        handler=tools.git_diff,
    )
    ctx.register_tool(
        name="grep_search",
        toolset="dev_tools",
        schema=schemas.GREP_SEARCH,
        handler=tools.grep_search,
    )
    ctx.register_tool(
        name="run_linter",
        toolset="dev_tools",
        schema=schemas.RUN_LINTER,
        handler=tools.run_linter,
    )
```

На этом plugin состоит из четырёх файлов:

```
plugin.yaml
schemas.py
tools.py
__init__.py
```

Это удобное разделение: manifest описывает plugin, схемы предназначены для LLM, handlers выполняют работу, а `register()` связывает их с Hermes. Такой же подход используется в официальном руководстве по созданию plugins.

Проверяем plugin через Doctor

Пока Hermes Desktop не перезапускаю.

В Terminal выполняю:

```
hermes plugins doctor .\.\hermes\plugins\dev-tools --ci
```

Я хочу получить успешную проверку без registration errors.

Plugin Doctor использует реальный механизм discovery, manifest parser и регистрацию Hermes, поэтому эта проверка значительно полезнее обычного запуска `python __init__.py`.

Если здесь появилась ошибка, дальше не продолжаю. Сначала исправляю указанную строку.

Разрешаем project-local plugins

Если Hermes Desktop был запущен в предыдущей практике из PowerShell с переменной:

```
HERMES_ENABLE_PROJECT_PLUGINS
```

она действует только пока этот процесс запущен.

Поэтому для чистой проверки я закрываю Hermes Desktop.

Открываю обычный PowerShell Windows.

Перехожу в проект:

```
Set-Location $projectRoot
```

Разрешаю project-local plugins:

```
$env:HERMES_ENABLE_PROJECT_PLUGINS="1"
```

Проверяю обнаружение:

```
hermes plugins list
```

Ищу:

```
dev-tools
```

Теперь включаю plugin:

```
hermes plugins enable dev-tools
```

Hermes plugins являются opt-in: обнаруженный plugin ещё не считается активным, пока он явно не включён.

Запускаем Hermes Desktop

Из того же PowerShell выполняю:

```
hermes desktop --cwd "$projectRoot"
```

Не запускаю Desktop отдельным ярлыком, потому что тогда переменная окружения текущего PowerShell может не попасть в процесс.

После открытия Hermes создаю новый чат.

Проверяем загрузку plugin

В чате ввожу:

```
/plugins
```

Ищу `dev-tools`.

Он должен быть загружен.

Теперь:

```
/tools list
```

Среди инструментов должны находиться:

```
git_diff  
grep_search  
run_linter
```

Команда `/tools` управляет инструментами текущей сессии. Если конкретный Tool отключён, его можно включить через `/tools enable <name>`.

Если наши инструменты отображаются выключенными, выполняю:

```
/tools enable git_diff
```

Затем:

```
/tools enable grep_search
```

И:

```
/tools enable run_linter
```

После изменения набора инструментов Hermes может перезапустить сессию. Это нормально, потому что для модели меняется список доступных функций.

Проверяем git_diff

Первый вызов делаем максимально явно.

В новом сообщении пишу:

Используй инструмент `git_diff`.

Покажи только незакоммиченные изменения файла `dev_lab.py`.
Ничего не изменяй.

В интерфейсе Hermes должна появиться карточка вызова:

```
git_diff
```

Я раскрываю её.

Проверяю аргументы.

Ожидаю примерно:

```
{  
  "path": "dev_lab.py",  
  "staged": false  
}
```

В результате должен присутствовать diff изменения функции `format_username`.

Сейчас модель не запускала терминал сама. Она сформировала структурированный вызов нашего инструмента, Hermes передал параметры handler, а handler выполнил только заранее разрешённую операцию `git diff`.

Проверяем staged diff

Теперь пишу:

Используй `git_diff` и проверь staged-изменения проекта.

Поскольку исходная версия `dev_lab.py` помещена в index как staged baseline, staged diff показывает добавление этой исходной версии относительно текущего `HEAD`. Рабочее изменение с `replace(" ", "_")` в staged diff пока не входит.

Ожидаемый аргумент:

```
{
  "staged": true
}
```

В ответе инструмента должно быть staged-добавление исходного `dev_lab.py`. После финального `git add` index будет обновлён до готовой версии, а единственный итоговый коммит зафиксирует весь результат практики.

Здесь хорошо видно, что одна функция работает в двух режимах через валидируемый параметр.

Проверяем grep_search

Теперь протестируем поиск.

Отправляю:

Используй `grep_search`.

Найди все использования `LegacyPaymentGateway` внутри `demo_codebase`.
Ничего не изменяй.

Я ожидаю вызов:

```
grep_search
```

с параметрами примерно:

```
{
  "pattern": "LegacyPaymentGateway",
  "path": "demo_codebase",
  "max_results": 30
}
```

В результате должны появиться файлы, где класс объявлен и используется.

Наш handler внутри вызывает:

```
git grep
```

Поэтому отдельный GNU `grep` на Windows нам не требуется.

Что важно знать про `git grep`

Этот вариант поиска работает по файлам, которые отслеживаются Git.

Если я создал совершенно новый файл, но ещё никогда не добавлял его в репозиторий, `git grep` может его не увидеть.

Для нашей учебной кодовой базы это нужно учитывать.

`git grep` работает по tracked-файлам рабочего дерева. Совершенно новый untracked-файл он не увидит, а содержимое рабочего файла может отличаться от staged-версии в index. Если какие-то файлы `demo_codebase` ещё не находятся под Git, я проверяю:

```
git status --short
```

и при необходимости выполняю `git add` для staged baseline без промежуточного коммита. Итоговая готовая версия всё равно будет зафиксирована единственным коммитом практики.

Сам инструмент при этом остаётся только читающим.

Проверяем linter

Теперь отправляю:

Используй `run_linter`.

Проверь файл:

```
demo_codebase/quality/bad_example.py
```

Ничего автоматически не исправляй.

Ожидаю вызов `run_linter`.

В результате Ruff должен сообщить о неиспользуемом импорте `os`.

Обычно мы увидим правило:

```
F401
```

Поле:

```
passed
```

будет иметь значение `false`.

Это не означает, что сам Tool сломался.

Наоборот, Tool успешно запустил линтер, а линтер обнаружил нарушение.

То есть нужно различать ошибку выполнения инструмента и отрицательный результат проверки.

Исправляем lint-проблему вручную

Теперь для чистоты эксперимента я сам даю Hermes отдельную команду:

```
Удалить неиспользуемый импорт os из:
```

```
demo_codebase/quality/bad_example.py
```

```
Других изменений не вносить.
```

После этого снова отправляю:

```
Используй run_linter и повторно проверь:
```

```
demo_codebase/quality/bad_example.py
```

Теперь ожидаю:

```
"passed": true
```

Это хороший пример цикла:

мы выполнили проверку;

получили диагностический результат;

изменили файл;

повторно запустили тот же Tool;

подтвердили устранение проблемы.

Проверяем валидацию пути

Теперь специально проверим одно из ограничений handler.

Отправляю:

Используй run_linter для пути:

..\..\

Только покажи результат инструмента.

Наш Tool не должен уходить выше:

корень проекта `hermes-multiagent`

Он должен вернуть сообщение:

Путь выходит за пределы проекта.

Это важный момент.

Ограничение существует не только в текстовом промпте. Оно непосредственно реализовано в Python-handler.

Проверяем валидацию grep

Теперь пишу:

Вызови grep_search с пустым поисковым шаблоном и покажи ответ инструмента.

Если модель действительно передаст пустой `pattern`, handler вернёт:

Параметр pattern не может быть пустым.

Если модель сама не отправит некорректный вызов из-за schema, это тоже хороший результат: ограничения параметров уже помогают ей сформировать правильный tool call.

Проверяем Tool без прямого указания его имени

До этого я специально говорил Hermes, какой инструмент использовать.

Теперь проверим выбор инструмента моделью.

Создаю новую сессию.

Пишу:

Проверь текущие незакоммиченные изменения файла `dev_lab.py`.

Мне нужно понять, что именно изменилось.

Ничего не изменяй.

Если описание `git_diff` составлено достаточно ясно, модель должна сама выбрать этот Tool.

Мне здесь важно показать студентам, зачем мы подробно писали `description`.

Мы не программировали условие:

если пользователь написал "изменения", вызвать `git_diff`

LLM сама сопоставляет задачу с описанием доступной функции.

Проверяем автоматический выбор grep

В новой задаче пишу:

Найди все места в кодовой базе, где используется старый платёжный gateway `LegacyPaymentGateway`.

Верни файлы и строки.

Ожидаю автоматический вызов:

`grep_search`

Если модель вместо этого пытается читать файлы вручную, я открываю `schemas.py` и ещё раз смотрю описание Tool.

Именно качество schema влияет на то, насколько очевидно для модели назначение инструмента.

Проверяем автоматический выбор линтера

Теперь пишу:

Проверь качество Python-кода в каталоге `demo_codebase`.

Мне нужны только результаты статического анализа.

Не исправляй код.

Ожидаю вызов:

```
run_linter
```

Параметром будет примерно:

```
{
  "path": "demo_codebase"
}
```

Ruff проверит сразу весь каталог.

Даём агенту комплексную задачу

Теперь проводим финальный эксперимент.

Я создаю новую сессию и отправляю обычный рабочий запрос без перечисления конкретных названий Tools:

Проведи read-only анализ текущего состояния проекта.

Сначала определи, какие незакоммиченные изменения есть в dev_lab.py.

Затем найди места использования LegacyPaymentGateway в demo_codebase.

После этого выполни статическую проверку Python-кода в demo_codebase.

Никакие файлы не исправляй и не изменяй.

В конце подготовь краткий отчёт по результатам всех проверок.

Теперь я наблюдаю за tool calls.

Ожидаю три специализированных вызова:

```
git_diff
grep_search
run_linter
```

Порядок может немного отличаться, если действия независимы, но все три должны быть использованы по назначению.

После каждого вызова результат возвращается LLM, и итоговый отчёт уже формируется на основании фактических данных инструментов.

Почему это лучше одного универсального terminal

Сейчас я хочу отдельно показать студентам архитектурную разницу.

Если дать модели только универсальный терминал, она может сама сформировать:

```
git diff
```

или команду поиска, или запуск Ruff.

Но вместе с этим она получает гораздо более широкую возможность выполнять другие команды.

Наш `git_diff` технически способен только читать diff.

`grep_search` способен только выполнять заранее определённый поиск.

`run_linter` способен только запустить Ruff без автоматического исправления.

Получается гораздо более узкая поверхность действий.

Это особенно полезно для агентов, которым нужен только анализ проекта.

Смотрим Tools текущей сессии

Теперь ввожу:

```
/tools list
```

Проверяю, что три инструмента всё ещё активны.

Hermes позволяет включать и отключать Tool непосредственно в текущей сессии через `/tools enable` и `/tools disable`. Отключение удаляет инструмент из доступного модели toolset.

Для эксперимента отключаю:

```
/tools disable run_linter
```

В новой сессии спрашиваю:

Каким доступным специализированным инструментом ты можешь проверить Python-код линтером?

`run_linter` больше не должен находиться среди доступных функций.

Возвращаю его:

```
/tools enable run_linter
```

Проверяем plugin через /plugins

Теперь ввожу:

```
/plugins
```

Здесь я могу проверить, что `dev-tools` действительно относится к загруженным plugins.

Это отличается от `/tools list`.

`/plugins` показывает уровень расширения Hermes.

`/tools list` показывает функции, которые доступны модели внутри конкретной сессии.

Один plugin может зарегистрировать несколько Tools, как в нашем случае.

Финальная проверка и фиксация

После удаления учебного неиспользуемого импорта повторно запускаю проверки:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe -m ruff check demo_codebase dev_lab.py --no-cache
hermes plugins doctor .\.hermes\plugins\dev-tools --ci
git diff --check
```

Убеждаюсь, что `.ruff_cache` не создан, а `rag/logs` и другие генерируемые файлы не попадают в Git.

Теперь обновляю staged baseline до итогового состояния и создаю единственный коммит объединённой практики:

```
git add dev_lab.py dev-requirements.txt demo_codebase\quality .hermes\plugins\dev-tools tests
git commit -m "Практики 2.5–2.6: добавить безопасные dev-tools"
```

После коммита `git status` должен показывать чистое рабочее дерево.

Что получилось в результате

В этой работе мы создали один Hermes plugin `dev-tools`.

Внутри него зарегистрированы три независимых инструмента.

`git_diff` получает состояние изменений репозитория.

`grep_search` выполняет контролируемый поиск по Git-кодовой базе.

`run_linter` запускает Ruff без изменения файлов.

При этом LLM не получает произвольную функцию вроде «выполни любую команду». Она видит три конкретные схемы с определёнными параметрами.

Когда возникает задача, модель выбирает подходящий Tool, формирует структурированные аргументы и передаёт их Hermes.

Hermes запускает handler.

Handler проверяет аргументы, выполняет строго ограниченную операцию и возвращает JSON.

Результат снова попадает в рабочий контекст модели.

После этого агент решает, достаточно ли информации или требуется следующий Tool.

Именно на этом небольшом примере мы видим практическое устройство инструментального агента: модель отвечает за выбор следующего действия, а Tools определяют, какие реальные действия система вообще способна безопасно выполнить.
